

[学习资源](#)

最佳实践

[Copy page](#)

Coding Agent 最佳实践：从 Prompt、Plan 到 Skills 与 Workflow 治理

随着大模型能力的提升，AI 编码工具正在从“代码补全助手”逐渐演变为能够参与完整开发流程的 **Coding Agent**。与传统 Copilot 类工具不同，Coding Agent 不再只是根据提示生成代码，而是可以读取代码库、修改文件、运行命令、调用外部工具，并在多轮交互中完成复杂任务。

在这一趋势下，开发者需要的不仅是“如何写提示词”，而是一套能够稳定使用 Coding Agent 的工作方法。目前主流的 Coding Agent 在设计上都逐渐收敛到相似的使用模式：通过明确任务上下文、规划执行步骤、沉淀项目规则、接入外部工具以及自动化重复流程，让 Agent 能够在真实开发环境中长期协作。

本文基于这些工具的官方实践，总结出一套更通用的 **Coding Agent 最佳实践框架**。

一、将 Coding Agent 视为协作者，而不是一次性助手

在使用 Coding Agent 时，一个常见误区是将其当作一次性问答工具：

提出一个问题，获得一段代码，然后结束对话。

然而，这种使用方式并不能发挥 Coding Agent 的真正能力。

它更适合作为一个可以持续配置和优化的“团队成员”。开发者可以通过项目说明文件、配置规则、工具接入和技能沉淀，不断改进 Agent 的行为，使其逐渐适应团队的开发流程。Claude Code 也采用了类似的设计理念。它不仅可以读取和修改代码库，还可以通过 Skills、Hooks、Subagents 等机制持续优化工作流程，使 Agent 的行为逐渐稳定下来。

二、任务输入结构化：上下文比提示技巧更重要

许多开发者在使用 Coding Agent 时会过度关注提示词技巧，而忽略了更重要的因素：**任务上下文**。

在复杂代码库中，一个有效的任务描述通常应包含四个要素：

▼ 目标 (Goal)

明确说明需要实现的功能或修改，例如修复 bug、实现接口或重构模块。

▼ 上下文 (Context)

提供相关代码文件、错误信息、文档或示例。例如说明涉及哪些文件、哪些函数或哪些模块。

▼ 约束 (Constraints)

列出需要遵守的工程规范，例如代码风格、架构规则、安全要求或依赖限制。

▼ 完成标准 (Done when)

说明任务完成的判断条件，例如测试通过、行为变化或 bug 不再复现。

这种结构化任务输入可以减少 Agent 的猜测空间，使其生成的修改更加稳定、可审查。

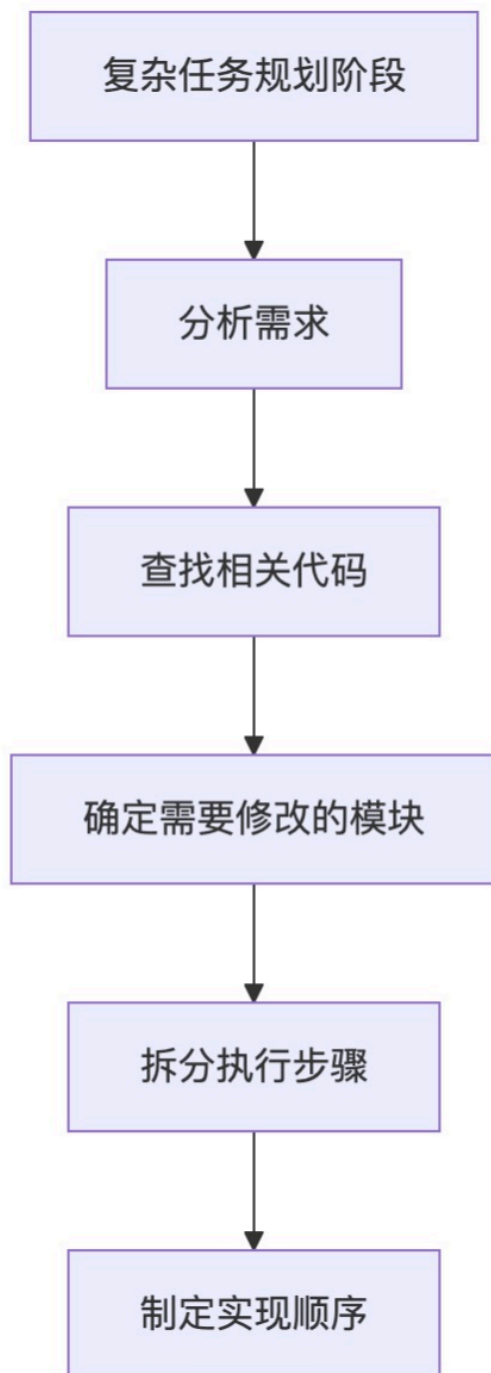
在大多数 Coding Agent 中，可以通过引用文件、提供代码片段等方式让 Agent 获取更准确的上下文，也可以通过 prompt 中明确列出这些信息来实现。

三、复杂任务应先规划，再执行

Coding Agent 与传统代码补全工具的一个关键区别在于：它能够处理多步骤任务。

对于复杂需求，直接让 Agent 编写代码往往会导致逻辑错误或反复修改。更有效的方式是让 Agent 先生成执行计划，再进入实现阶段。

这一规划阶段通常包括：



Claude Code 的工作流中鼓励在复杂任务中先进行分析和规划，例如探索代码结构、确认修改范围，再开始实现。有的 Coding Agent 则提供了专门的 Plan 模式，可以在实现之前生成完整执行计划。

通过这种方式，Agent 可以从“即时生成代码”转变为“按照计划逐步完成任务”。

四、将重复规则沉淀为项目级配置文件

在实践中，许多提示词其实是在重复说明项目规则，例如：

- 项目目录结构
- 构建命令
- 测试流程
- 代码规范
- PR 提交流程

如果这些规则每次都写在 prompt 中，不仅效率较低，也容易随着对话变化而产生不一致。

因此，大多数 Coding Agent 都提供了一种机制，用于将这些 **长期有效的项目规则** 写入项目级配置文件，使 Agent 在执行任务时能够自动加载相关上下文。

在一些工具中，这类配置文件通常表现为面向 Agent 的项目说明文件，例如用于描述代码仓库结构、运行方式以及开发规范。也有一些系统通过配置、脚本或 workflow 机制来存储这些规则，使 Agent 在不同会话中仍然能够遵循统一的项目约束。

无论具体实现形式如何，其核心目标都是一致的：将原本需要在对话中反复说明的信息，沉淀为 **稳定的项目上下文**。

✓ 从实践角度看，可以总结为一条简单原则：**临时指令写在 prompt 中，长期规则写入项目级配置文件。**

五、执行环境决定了 Agent 的能力

在实际使用 Coding Agent 的过程中，开发者经常会将效果不稳定归因于模型能力，但很多问题的根源实际上来自 **执行环境配置不完整**。

与传统的代码补全工具不同，Coding Agent 通常需要在真实开发环境中执行一系列操作，例如：



BigModel

- 读取和修改代码文件
- 运行构建或测试命令
- 调用外部工具或 API
- 与版本控制系统交互



因此，Agent 的行为不仅取决于模型能力，也取决于其 **运行环境是否完整且可访问**。如果环境配置不当，Agent 很容易出现以下问题：



- 无法定位正确的项目目录
- 没有权限读取或修改关键文件
- 无法运行构建或测试命令
- 无法访问外部工具或服务

这些问题往往表现为“模型理解错误”或“生成代码质量差”，但本质上是 **Agent 无法获得足够的执行能力或上下文信息**。

目前主流的 Coding Agent 工具通常都会提供一套 **环境配置机制**，用于定义 Agent 在项目中的行为边界。例如：

- 指定默认模型或推理强度
- 控制文件读写权限与沙箱策略
- 定义允许执行的命令
- 配置外部工具或服务连接

虽然不同工具在具体实现上有所区别，但总体目标是相同的：为 Agent 提供一个 **稳定、可控且可重复的执行环境**。

在实践中，开发者通常需要重点关注以下几类配置：

- 项目工作目录与代码访问权限
- 可执行命令范围（如 `build`、`test`、`lint`）
- 外部工具或数据源连接
- 团队统一的默认行为设置

✓ 从更宏观的角度看，Coding Agent 的运行依赖于三类上下文：

- **任务上下文 (Task Context)** ：当前任务的 prompt 与输入信息
- **项目上下文 (Project Context)** ：代码仓库结构与工程规则
- **环境上下文 (Environment Context)** ：工具、权限与执行环境

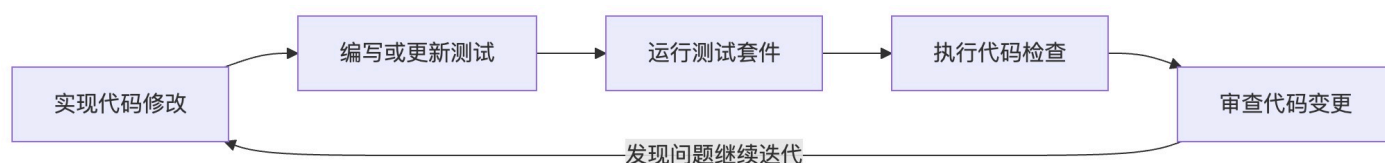
其中，环境上下文决定了 Agent 能够做什么，以及能够做到什么程度。

六、让 Coding Agent 参与完整开发闭环

如果 Coding Agent 只被用于生成代码，其价值会大大降低。在真实的软件开发过程中，一段代码是否能够被接受，往往取决于它是否通过测试、符合工程规范，并且经过必要的代码审查。

因此，更合理的使用方式是让 Coding Agent 参与 **完整的开发闭环 (Development Loop)**，而不仅仅是代码生成。

一个典型的 Agent 开发循环通常包括以下步骤：



1. **实现代码修改** 根据任务需求修改或新增代码。
2. **编写或更新测试** 为新功能或修复的缺陷补充测试用例。
3. **运行测试套件** 执行单元测试或集成测试，验证修改是否正确。
4. **执行代码检查** 运行 lint、格式检查或类型检查，确保代码符合工程规范。
5. **审查代码变更** 检查 diff，识别潜在问题、回归风险或异常修改。

在这一流程中，Coding Agent 不再只是一个“代码生成器”，而是能够参与 **实现、验证和审查** 的开发协作节点。

当前主流的 Coding Agent 工具普遍支持这一工作模式。例如，一些工具允许 Agent 在修改代码后自动运行测试或构建命令，也可以在代码变更后执行检查脚本或触发自动审查流程。通过这些机制，Agent 可以在本地开发环境或 CI 流程中持续参与代码质量保障。

- ✓ 从开发流程的角度看，这种模式将 Coding Agent 从传统的 **代码生成工具 (code generator)** 转变为 **开发循环中的执行单元 (execution node)**。

当 Agent 能够参与整个开发闭环时，其作用不仅是提高编写代码的效率，还可以在测试、验证和审查环节减少人工重复操作，从而提升整体开发效率。

七、通过 MCP 扩展 Agent 的上下文

在实际开发过程中，Agent 所需的信息并不总是存在于代码仓库内部。很多影响开发决策的重要上下文，往往分散在外部系统中，例如：

- Issue 与需求管理系统
- CI/CD 运行状态
- 数据库结构或线上数据
- API 文档与外部服务说明

如果这些信息每次都依赖人工复制粘贴，不仅效率较低，也会使上下文传递变得零散且不稳定。对于需要持续执行多步骤任务的 Coding Agent 来说，这种方式很难支撑复杂 workflow。

因此，许多 Coding Agent 工具支持 Model Context Protocol (MCP)，用于连接外部工具和系统，使 Agent 能够在代码仓库之外，直接访问任务所需的实时信息。

通过 MCP，Agent 可以访问的对象通常包括：

- 代码托管与协作平台
- 数据库与查询接口
- API 服务与技术文档
- 团队内部工具与自动化系统

这意味着，Coding Agent 获取信息的方式，开始从“依赖人工描述”转向“依赖工具直接查询”。



从工作流角度看，这一变化非常关键。当 Agent 只能使用 prompt 中提供的信息时，它处理的往往只是局部任务；而当它能够连接外部系统时，才能真正参与到更完整的开发流程中，例如读取 issue 背景、检查 CI 失败原因、查询接口定义，或结合数据库结构分析问题来源。

因此，接入外部工具的意义，并不只是“让 Agent 多了几个工具”，而是在于它扩展了 Agent 的上下文边界，使其能够从 **代码仓库内的执行者**，进一步转变为 **面向真实研发环境的协作节点**。

八、将重复流程沉淀为 Skills

在长期使用 Coding Agent 的过程中，开发者往往会发现某些任务会反复出现。例如：

- PR 审查
- 日志分析
- 发布说明生成
- 标准调试流程

如果每次都通过 prompt 手动描述这些任务，不仅会造成大量重复输入，也容易因为描述不一致而影响结果稳定性。

因此，许多 Coding Agent 系统都提供了 **Skill（技能）机制**，用于将常见工作流程封装为可复用的执行模板。

从本质上看，Skill 可以理解作为一种 **结构化任务模板**。它将原本分散在 prompt 中的执行逻辑进行抽象，使 Agent 在面对类似任务时能够自动应用同一套处理流程。



关于 Skill 的概念以及如何编写 Skill，可以参考 [火爆社区的 Claude Skill 到底是什么？从创建到优化的终极指南](#)。

不同 Coding Agent 工具会采用不同方式来管理技能。例如，有些工具使用独立的技能文件来定义任务模板，也有工具允许通过配置或脚本机制注册可复用工作流。

虽然实现形式不同，但其核心目标是一致的：**将零散的提示词经验沉淀为可复用的工作流能力**。

在实践中，可以通过一个简单的经验法则来判断是否需要创建 Skill：



通过这种方式，Coding Agent 的使用方式会逐渐从“即时对话驱动”转变为“基于工作流的任务执行”。随着技能库不断积累，Agent 的行为也会更加稳定和可预测。

九、稳定流程可以进一步自动化

当某个 Skill 已经能够稳定执行时，就可以进一步将其自动化运行。

在长期开发过程中，许多任务具有明显的周期性或重复性。例如：

- 定期生成 commit 总结
- 自动检查 CI 失败原因
- 扫描潜在 bug 或异常日志
- 生成开发日报或周报

如果这些任务仍然需要开发者手动触发，即使已经封装为 Skill，也仍然会产生大量重复操作。

因此，许多 Coding Agent 系统都提供了 **自动化（Automation）机制**，允许 Agent 在指定时间或条件下自动执行任务。

- ❗ Automation 可以被理解为 Skill 的下一层能力。Skill 定义了任务的执行方法，而 Automation 则决定任务何时被触发以及如何持续运行。

例如，一个用于生成发布说明的 Skill 可以被配置为：

- 在每次版本发布时触发
- 每周自动生成一次发布总结
- 在 CI 完成后自动运行

通过这种方式，Coding Agent 可以在后台持续执行任务，而不需要开发者每次手动启动。这一机制的意义在于，它使 Coding Agent 的角色从 **交互式工具（interactive tool）**，逐渐转变为 **持续运行的开发辅助系统（continuous assistant）**。

十、合理管理 Agent 会话

在使用 Coding Agent 时，会话不仅仅是简单的聊天记录。它实际上是一条持续积累上下文信息、推理过程和执行结果的 **工作线程**。

随着任务不断推进，Agent 会在同一会话中逐步积累：

- 任务目标
- 相关代码上下文
- 已执行的修改
- 推理过程与中间决策

这些信息共同构成了 Agent 当前任务的 **上下文状态**。

如果会话管理不当，例如在同一线程中不断叠加多个无关任务，就可能导致上下文过于复杂，从而影响 Agent 的判断与执行质量。因此，在实际使用 Coding Agent 时，合理的会话管理策略非常重要。常见实践包括：

▼ 每个任务使用独立线程

避免在同一会话中混合多个任务，保持上下文清晰。

▼ 避免线程过

当会话积累了大量历史信息时，可以通过摘要或压缩方式减少上下文负担。

▼ 分支任务创建新线程

如果任务出现新的探索方向，可以在新的线程中继续，而不是在原线程中不断叠加修改。

▼ 定期压缩历史上下文

在更复杂的开发场景中，还可以引入 **多 Agent 协作模式**。例如，将探索代码结构、运行测试或排查问题等子任务交由独立 Agent 处理，而主 Agent 负责整体任务协调。通过这种方式，可以在保持主任务上下文清晰的同时，提高复杂任务的执行效率。

会话管理实际上是一种 **上下文治理（context management）机制**。良好的会话结构能够帮助 Coding Agent 在多轮任务中保持清晰的推理路径，从而提升整体稳定性与执行效果。

结语

Coding Agent 的能力不仅来自模型本身，更来自于开发者如何设计其使用方式。

在实践中，一个成熟的 Coding Agent 工作流程通常包括：



通过这一流程，Coding Agent 可以逐渐从简单的代码生成工具，演变为能够参与完整软件开发周期的协作系统。

📖 学习资源



Coding Agent 工作原理



Agentic 扩展组件



记忆机制



常用 workflow

>
